

Document number	P3806R0
Date	2025-08-02
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::cycle

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes adding `views::cycle`, a Tier 1 range adaptor as described in [P2760](#), to enhance the C++29 Ranges library by enabling infinite repetition of a range's elements.

Revision history

R0

Initial revision.

Discussion

There is currently no standard range adaptor in C++ that allows repeating a range endlessly. Although similar behavior can be approximated via `views::repeat(r) | views::join` or a custom `generator`, such constructs are limited to forward ranges and often introduce additional complexity and boilerplate. Additionally, they lack the semantic clarity and composability offered by a dedicated adaptor.

The ability to cycle through elements infinitely is a common requirement in many domains: circular buffers, animations, event loops, and more. This functionality exists natively in other modern languages (e.g., Python's `itertools.cycle`, Rust's `.cycle()`), highlighting a gap in C++'s otherwise powerful Ranges library.

We propose introducing `views::cycle` as a simple, intuitive, and efficient adaptor for producing infinite, multi-pass ranges that cycle through the original range. This fills a notable gap in the existing standard and improves parity with other languages while supporting both eager and lazy range pipelines:

```
for (auto&& song : playlist | views::cycle | views::take(100)) {
    play(song);
}
```

Design

Class signature

Since `views::cycle` repeats the original range, the minimum requirement is `forward_range` to ensure multi-pass. The proposed signature of `cycle_class` is:

```
template<view V>
    requires forward_range<V>
    class cycle_view : public view_interface<cycle_view<V>> {
```

```
// ...
};
```

Handling of Empty Ranges

range/v3's cycle view (which is named `cycled_view`) requires the original range to be [non-empty](#):

```
explicit cycled_view(Rng rng)
: rng_(std::move(rng))
{
    RANGES_EXPECT(!ranges::empty(rng_));
}
```

It always expects a non-empty range and produces an infinite range. This requires the user to ensure the input range is non-empty, or else the behavior is undefined:

```
auto e = ranges::views::empty<int>;
auto c1 = e | ranges::views::cycle;    // UB

vector<int> v;
auto c2 = v | ranges::views::cycle;    // also UB
```

Unlike range-v3's `views::cycle`, the proposed `views::cycle` supports empty ranges. This is consistent with other standard range adaptors that naturally handle empty input by producing empty views. Supporting empty ranges avoids undefined behavior in common scenarios, such as default-initializing a `cycle_view`:

```
auto c = vector{42} | views::cycle;
decltype(c) c2{};    // Default construction is now well-formed
```

This improves composability by aligning with the standard library's treatment of empty ranges, yielding intuitive and consistent behavior. Consequently, since a `cycle_view` can now be empty, its `end()` cannot be `unreachable_sentinel` as in range-v3. Instead, we use `default_sentinel`, with the following semantics:

- If the base range is empty, `begin() == end()` and the `cycle_view` is empty.
- If the base range is non-empty, iteration proceeds infinitely, and `end()` is never reached.

This approach preserves correctness for both empty and infinite cases without introducing special-case logic.

End Iterator Is Not Cached

For bidirectional operations such as `operator--()`, when we find that the current iterator has reached the beginning of the original range, we need to move the current iterator back to the last element of the original range:

```
if (current_ == ranges::begin(base_)) {
    current_ = end-iterator-of-base_; // Repositioning
    // ...
}
--current_;
```

However, getting the end iterator of the original range may not be constant time unless the original range is `common_range` or models both `sized_range` and `random_access_range`; in such case, we can extract them as we do in `cartesian-common-arg-end`:

```
template<cartesian-product-common-arg R>
constexpr auto cartesian-common-arg-end(R& r) {    // exposition only
    if constexpr (common_range<R>) {
        return ranges::end(r);
    } else {
        return ranges::begin(r) + ranges::distance(r);
    }
}
```

However, it is worth noting that range/v3 takes a more aggressive approach, providing bidirectional operations as long as the underlying range models `bidirectional_range`, and providing random-access operations as long as the underlying range models `random_access_range`.

For unsized ranges, random-access operation requires knowing the actual size to locate the correct position. In this case, range/v3's `cycled_view` will *eagerly* calculate the end iterator of the original range and [cache](#) it internally, so that it can be used to compute the size next time:

```
const char* s = /* */;
ranges::subrange rng(s,null_sentinel{});
static_assert(!ranges::sized_range<decltype(rng)>);

auto cycled = rng | ranges::views::cycle;
auto it = cycled.begin() + 42;    // O(n) time to get the rng's end iterator for computing size
```

Similarly, for bidirectional operations, range-v3 will also eagerly cache the end iterator if the underlying range is not [common_range](#).

Unlike range/v3, we deliberately avoid this optimization to prevent hidden costs and semantic surprises. Instead, we propose to restrict such operations only when the underlying range natively supports them, that is, when it is a [random_access_range](#) and [sized_range](#) for random-access, or a [bidirectional_range](#) and [common_range](#) for bidirectional support.

This avoids hidden $O(n)$ complexity, preserves lazy evaluation, reduces internal state, and eliminates the need for caching, consistent with the design philosophy of other standard views.

Not support `iter_swap`

`cycle_view` does not provide `iter_swap`, even if the underlying range is indirectly swappable. Although each iterator in a `cycle_view` can be dereferenced to access elements of the base range, those iterators may alias the same position in the base range due to cycling behavior:

```
vector v = {1, 2, 3};
auto cycled = v | views::cycle;

auto it1 = cycled.begin();           // points to v[0]
auto it2 = next(cycled.begin(), 3);  // also points to v[0] due to cycling

iter_swap(it1, it2);
// This looks like a swap between the first and fourth elements,
// but it's actually swapping v[0] with itself
```

Allowing `iter_swap` in such a context could lead to misleading or self-aliased operations, which violate user expectations and introduce semantic ambiguity. To avoid this, `cycle_view::iterator` does not provide `iter_swap` specialization.

difference_type Considerations

To support operations such as iterator difference or positioning, `cycle_view::iterator` needs to track how many full passes have been made through the base range. This is typically implemented by maintaining a counter `n`, which is incremented each time the iterator loops back to the beginning of the base range.

Although `n` can be represented using the `difference_type` of the underlying range, since the number of cycles itself only needs to count a finite number of iterations, for `difference_type` of `cycle_view::iterator`, the original `difference_type` may not be a good choice.

The reason is that the logical distance between two `cycle_view::iterators` may become much larger than any value that can be stored in the original `difference_type`; in such cases, subtracting them will result in overflow which leads to UB.

As a result, we propose using an integral-class type to represent the `difference_type` of `cycle_view::iterator`, that is, an implementation-defined integer type that is sufficiently wide to accommodate the expected range of iteration.

Not support `views::cycle(5)`

We do not propose supporting an overload of `views::cycle` that takes an integral value, such as `views::cycle(5)`, as a way to repeat a value or cycle a range a fixed number of times. While superficially convenient, such a form introduces semantic ambiguity and conflicts with established conventions in the Ranges library.

In particular, `views::cycle` already supports the following two idiomatic forms:

```
rng | views::cycle           // Pipe syntax
views::cycle(rng)           // Function-call syntax
```

These forms make it clear that `views::cycle` is a range adaptor closure object: it takes a range and returns a new view that conceptually repeats the elements of the input range infinitely.

Introducing something like `views::cycle(5)` could lead users to misinterpret it as `views::cycle(views::single(5))`, i.e., an infinite repetition of a single value. And those who wish to repeat a range exactly `n` times can compose `cycle` and `take` in more explicit way like `views::cycle(rng) | views::take(ranges::distance(rng) * n)`.

As such, this proposal deliberately omits such usage, consistent with `range/v3`.

Not borrow range

`cycle_view` cannot be a `borrowed_range`, because its iterators internally store a pointer to the `cycle_view` in order to access the stored underlying range.

Specifically, the iterator relies on calling `ranges::end` on underlying range to determine whether it has reached the end of a cycle. As such, `cycle_view` cannot be a `borrowed_range`, because the validity of its iterators depends on the lifetime of the view object.

Implementation experience

The author implemented `views::cycle` based on `libstdc++`, see [here](#).

Proposed change

This wording is relative to [latest working draft](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define cpp_lib_ranges_cycle 2025XXL // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
#include <compare>           // see \[compare.syn\]
#include <initializer_list>   // see \[initializer.list.syn\]
#include <iterator>           // see \[iterator.synopsis\]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified enumerate = unspecified; }

    // \[range.cycle\], cycle view
    template<view>
    requires forward_range<V>
    class cycle_view;

    namespace views { inline constexpr unspecified cycle = unspecified; }
    [...]
}
```

3. Add **25.7.? Cycle view** [\[range.cycle\]](#) after 25.7.24 [\[range.enumerate\]](#) as indicated:

-1- A cycle view presents an infinite view that endlessly repeats the source range.

-2- The name `views::cycle` denotes a range adaptor object ([\[range.adaptor.object\]](#)). Given a subexpression `E`, the expression `views::cycle(E)` is expression-equivalent to `cycle_view<views::all_t<decltype(E)>>{E}`.

-3- *[Example 1:*

```
auto cycle = views::iota(0, 3) | views::cycle;
println("{} ", cycle | views::take(10)); // prints [0, 1, 2, 0, 1, 2, 0, 1, 2, 0]
```

— end example]

[25.7.?.2] Class template `cycle_view` [range.cycle.view]

```
namespace std::ranges {
    template<view V>
        requires forward_range<V>
    class cycle_view : public view_interface<cycle_view<V>> {
    private:
        V base_ = V(); // exposition only

        // [range.cycle.iterator], class template cycle_view::iterator
        template<bool Const>
        class iterator; // exposition only

    public:
        cycle_view() requires default_initializable<V> = default;
        constexpr explicit cycle_view(V base);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple_view<V>)
        { return iterator<false>(*this, ranges::begin(base_)); }

        constexpr auto begin() const requires forward_range<const V>
        { return iterator<true>(*this, ranges::begin(base_)); }

        constexpr auto end() const noexcept { return default_sentinel; }
    };

    template<class R>
        cycle_view(R&&) -> cycle_view<views::all_t<R>>;
}
```

constexpr explicit cycle_view(V base);

-1- *Effects*: Initializes `base_` with `std::move(base)`.

[25.7.?.3] Class `cycle_view::iterator` [range.cycle.iterator]

```
namespace std::ranges {
    template<view V>
    class cycle_view<V>::iterator {
        using Parent = maybe_const<Const, cycle_view>; // exposition only
        using Base = maybe_const<Const, V>; // exposition only

        iterator_t<Base> current_ = iterator_t<Base>(); // exposition only
        Parent* parent_ = nullptr; // exposition only
        range_difference_t<Base> n_ = 0; // exposition only

        constexpr iterator(Parent& parent, iterator_t<Base> current); // exposition only

    public:
        using iterator_concept = see below;
        using iterator_category = see below;
        using value_type = range_value_t<Base>;
        using difference_type = see below;

        iterator() requires default_initializable<iterator_t<Base>> = default;

        constexpr iterator(iterator<!Const> i)
            requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

        constexpr iterator_t<Base> base() const;

        constexpr decltype(auto) operator*() const { return *current_; }

        constexpr iterator_t<Base> operator->() const
            requires has_arrow<iterator_t<Base>>;

        constexpr iterator& operator++();
        constexpr iterator operator++(int);
    };
}
```

```

constexpr iterator& operator--()
    requires bidirectional-common<Base> || sized-random-access-range<Base>;
constexpr iterator operator--(int)
    requires bidirectional-common<Base> || sized-random-access-range<Base>;

constexpr iterator& operator+=(difference_type n)
    requires sized-random-access-range<Base>;
constexpr iterator& operator-=(difference_type n)
    requires sized-random-access-range<Base>;

constexpr decltype(auto) operator[](difference_type n) const
    requires sized-random-access-range<Base>
{ return *(&this + n); }

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires sized-random-access-range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires sized-random-access-range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires sized-random-access-range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>> &&
        sized_range<Base>;

friend constexpr range_value_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
};
}

```

-1- `iterator::iterator_concept` is defined as follows:

- (1.1) — If *Base* models *sized-random-access-range*, then `iterator_concept` denotes `random_access_iterator_tag`.
- (1.2) — Otherwise, if *Base* models *bidirectional-common*, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (1.3) — Otherwise, `iterator_concept` denotes `forward_iterator_tag`.

-2- `iterator::iterator_category` is defined as follows:

- (2.1) — Let *C* denote `iterator_traits<iterator_t<Base>>::iterator_category`.
- (2.2) — If *C* models `derived_from<random_access_iterator_tag>` and *Base* models *sized_range*, `iterator_category` denotes `random_access_iterator_tag`.
- (2.3) — Otherwise, if *C* models `derived_from<bidirectional_iterator_tag>` and *Base* models *common_range*, `iterator_category` denotes `bidirectional_iterator_tag`.
- (2.4) — Otherwise, `iterator_category` denotes `forward_iterator_tag`.

-3- `iterator::difference_type` is an implementation-defined signed-integer-like type.

-4- *Recommended practice*: `iterator::difference_type` should be the smallest signed-integer-like type that is sufficiently wide to store the product of `range_difference_t<Base>` with itself, if such a type exists.

```
constexpr iterator(Parent& parent, iterator_t<Base> current);
```

-5- *Effects*: Initializes `current_` with `std::move(current_)` and `parent_` with `addressof(parent)`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

-6- *Effects*: Initializes `current_` with `std::move(i.current_)`, `parent_` with `i.parent_`, and `n_` with `i.n_`.

```
constexpr iterator_t<Base> base() const;
```

-7- *Effects*: Equivalent to: `return current_;`

```
constexpr iterator_t<Base> operator->() const
    requires has_arrow<iterator_t<Base>>;
```

-8- *Effects*: Equivalent to: `return current_;`

```
constexpr iterator& operator++();
```

-9- *Effects*: Equivalent to:

```
    if (++current_ == ranges::end(parent_>base_)) {
        current_ = ranges::begin(parent_>base_);
        ++n_;
    }
    return *this;
```

```
constexpr iterator operator++(int);
```

-10- *Effects*: Equivalent to:

```
    auto tmp = *this;
    ++*this;
    return tmp;
```

```
constexpr iterator& operator--()
    requires bidirectional_common<Base> || sized_random_access_range<Base>;
```

-11- *Effects*: Equivalent to:

```
    if (current_ == ranges::begin(parent_>base_)) {
        if constexpr (common_range<Base>)
            current_ = ranges::end(parent_>base_);
        else
            current_ = ranges::begin(parent_>base_) + ranges::distance(parent_>base_);
        --n_;
    }
    --current_;
    return *this;
```

```
constexpr iterator& operator--(int)
    requires requires bidirectional_common<Base> || sized_random_access_range<Base>;
```

-12- *Effects*: Equivalent to:

```
    auto tmp = *this;
    --*this;
    return tmp;
```

```
constexpr iterator& operator+=(difference_type n)
    requires sized_random_access_range<Base>;
```

-13- *Effects*: Equivalent to:

```

const auto first = ranges::begin(parent_>base_);
const auto dist = ranges::distance(parent_>base_);
const auto offset = current_ - first;
const auto new_n = n + offset;
const auto new_offset = new_n % dist;
n_ += new_n / dist;
current_ = first + range_difference_t<Base>(new_offset >= 0 ? new_offset : new_offset + dist);
return *this;

```

```

constexpr iterator& operator--(difference_type x)
requires sized-random-access-range<Base>;

```

-14- *Effects*: Equivalent to: `return *this -= x;`

```

friend constexpr bool operator==(const iterator& x, const iterator& y);

```

-15- *Returns*: `x.n_ == y.n_ && x.current_ == y.current_`.

```

friend constexpr bool operator==(const iterator& x, default_sentinel_t);

```

-16- *Returns*: `ranges::empty(x.parent_>base_)`.

```

friend constexpr bool operator<(const iterator& x, const iterator& y)
requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
requires random_access_range<Base>;

```

-17- Let *op* be the operator.

-18- *Effects*: Equivalent to:

```

if (x.n_ != y.n_)
    return x.n_ op y.n_;
return x.current_ op y.current_;

```

```

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
requires random_access_range<Base> &&
three_way_comparable<iterator_t<Base>>;

```

-19- *Effects*: Equivalent to:

```

using R = compare_three_way_result_t<iterator_t<Base>>;
if (x.n_ != y.n_)
    return R(x.n_ <=> y.n_);
return R(x.current_ <=> y.current_);

```

```

friend constexpr iterator operator+(const iterator& i, difference_type n)
requires sized-random-access-range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
requires sized-random-access-range<Base>;

```

-20- *Effects*: Equivalent to:

```

auto r = i;
r += n;
return r;

```

```

friend constexpr iterator operator-(const iterator& i, difference_type n)
requires sized-random-access-range<Base>;

```

-21- *Effects*: Equivalent to:

```

auto r = i;
r -= n;
return r;

```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>> &&
        sized_range<Base>;
```

-22- *Effects*: Equivalent to:

```
const auto dist = ranges::distance(x.parent_ -> base_);
return (x.n_ - y.n_) * dist + x.current_ - y.current_;
```

```
friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
```

-23- *Effects*: Equivalent to: `return ranges::iter_move(i.current_);`

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>